TAD - Fila

Prof. Fernando Garcia

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Filas

- Lista linear em que todas as inserções são realizadas em um extremo da lista e todas as retiradas e (e os acessos) são realizados no outro extremo da lista.

- O modelo intuitivo de uma fila:
  - fila de espera em que as pessoas no início da fila são servidas primeiro e as pessoas que chegam entram no fim da fila.

Filas

- São chamadas listas **fifo** (“first-in”, “first-out”).
- Existe uma ordem linear para filas que é a “ordem de chegada”.
- São utilizadas para processar itens de acordo com a ordem “primeiro-que-chega, primeiro-atendido”.

Filas

- Sistemas operacionais utilizam filas para regular a ordem na qual tarefas (processos) devem receber processamento e recursos devem ser alocados a processos.

TAD Filas

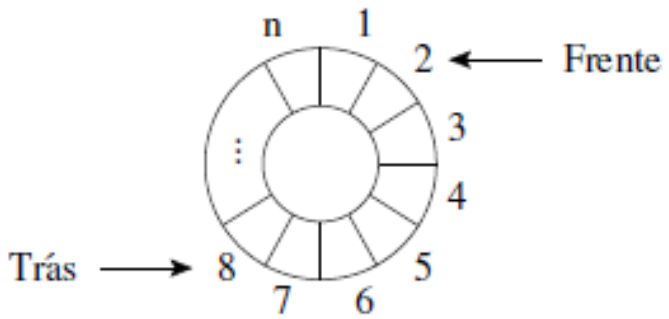
- Operações básicas:
  - Criar uma fila vazia.
  - Enfileirar o item x no final da fila.
  - Desenfileirar: retorna o item x no início da fila e o retira da fila.
  - Verificar se a fila está vazia. Essa função retorna *true* se a fila está vazia; do contrário, retorna *false*.

Implementação de Filas por meio de Arranjos

- Os itens são armazenados em posições contíguas de memória.
- A operação enfileira faz a parte de trás da fila expandir-se.
- A operação desenfileira faz a parte da frente da fila contrair-se.

Implementação de Filas por meio de Arranjos

- Imaginar o arranjo como um círculo. A primeira posição segue a última.
- Para enfileirar, basta mover o apontador (objeto) ***tras*** uma posição no sentido horário.
- Para desenfileirar, basta mover o apontador (objeto) ***frente*** uma posição no sentido horário.

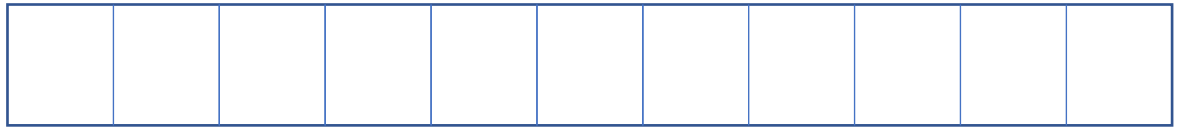


Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Propriedade das Filas

Fila

Fila Vazia



frente
tras

Propriedade das Filas

Fila

Enfileira

X_1										

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2									

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3								

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3	X_4							

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3	X_4	X_5						

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3	X_4	X_5	X_6					

frente tras

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

frente tras

Propriedade das Filas

Fila

Desenfileira

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

frente tras

Propriedade das Filas

Fila

Desenfileira

	X_2	X_3	X_4	X_5	X_6	X_7				

frente tras

Propriedade das Filas

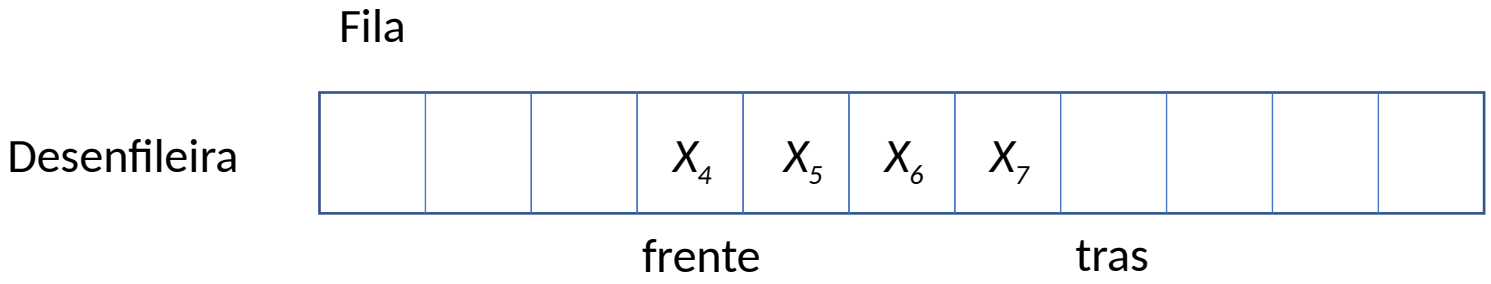
Fila

Desenfileira

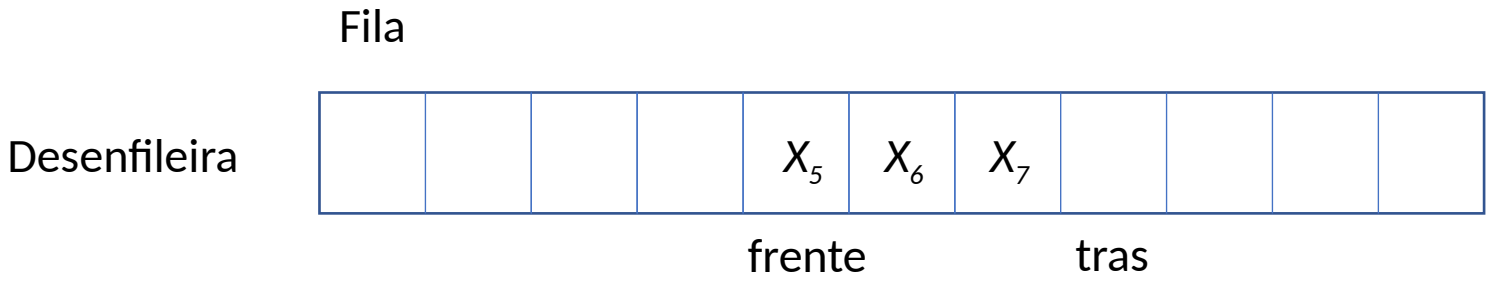
		X_3	X_4	X_5	X_6	X_7				

frente tras

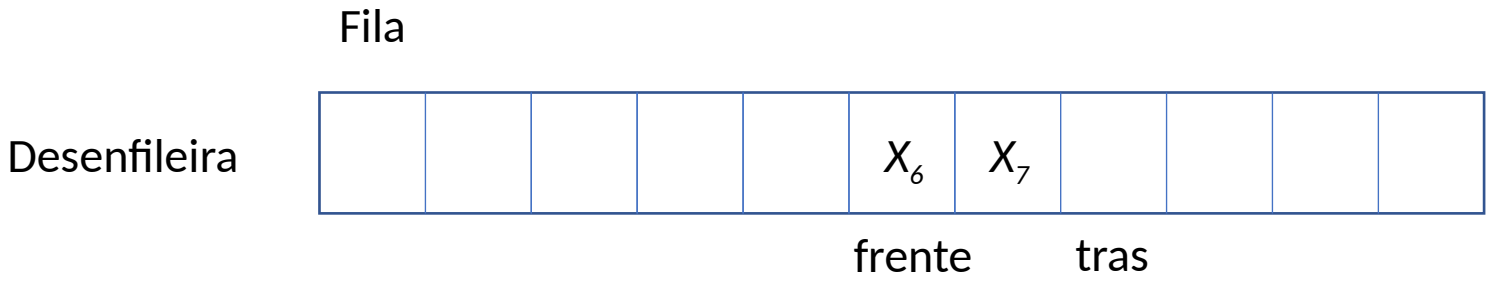
Propriedade das Filas



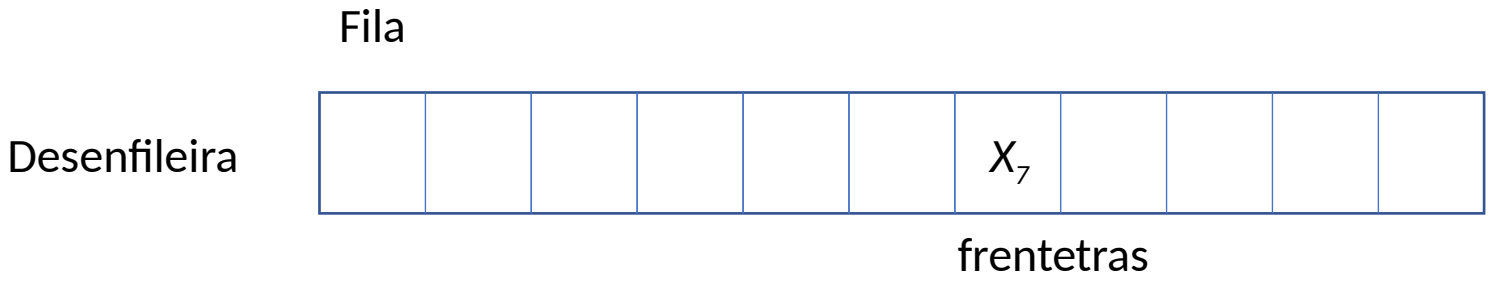
Propriedade das Filas



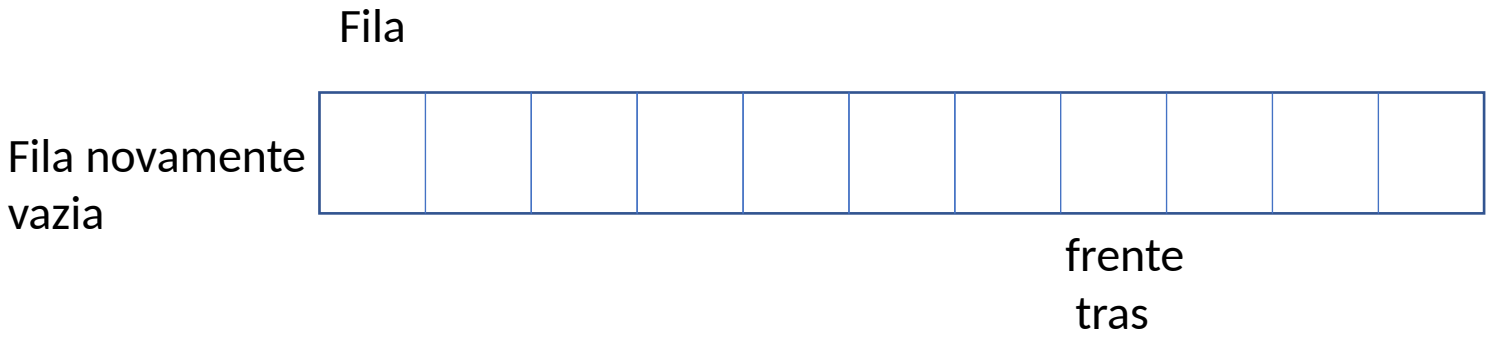
Propriedade das Filas



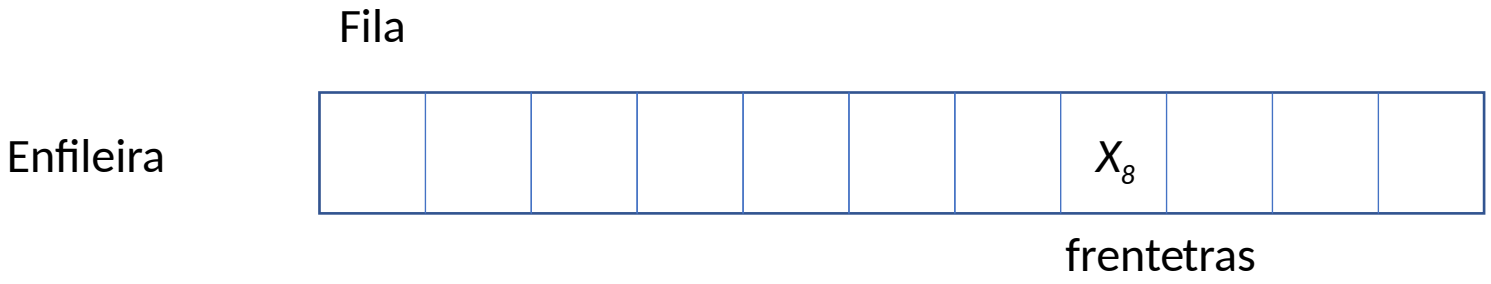
Propriedade das Filas



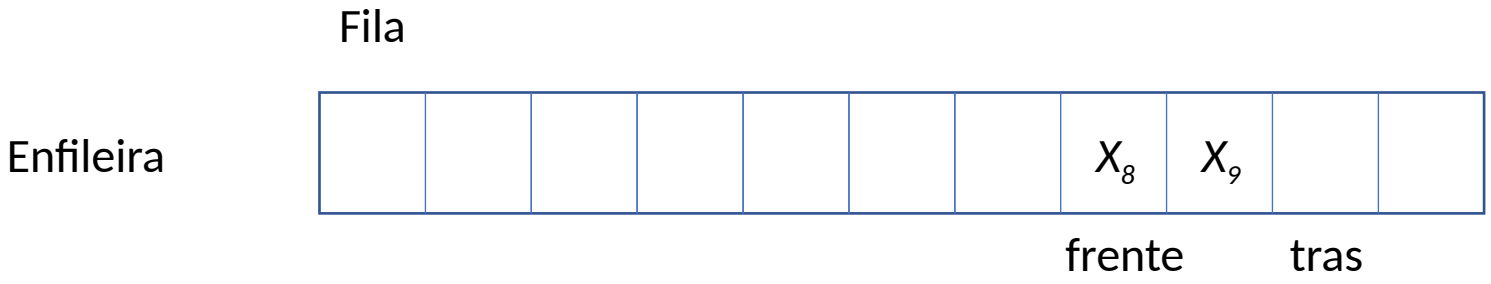
Propriedade das Filas



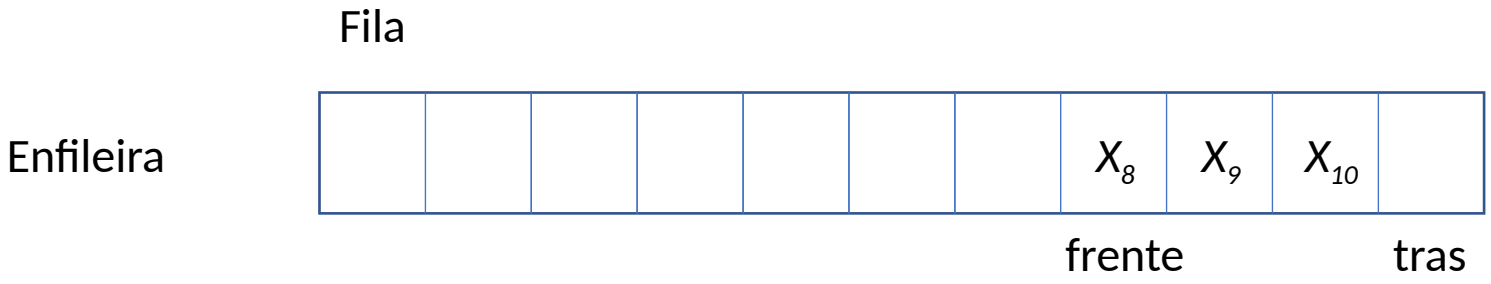
Propriedade das Filas



Propriedade das Filas



Propriedade das Filas



Propriedade das Filas

Fila

Enfileira

							X_8	X_9	X_{10}	X_{11}

tras frente

Propriedade das Filas

Fila

Enfileira

X_{12}							X_8	X_9	X_{10}	X_{11}

tras frente

Propriedade das Filas

Fila

Enfileira

X_{12}	X_{13}						X_8	X_9	X_{10}	X_{11}

tras frente

A implementação utiliza aritmética modular para enfileirar e desenfileirar. A posição "$n - 1$" do arranjo, apenas, é irrelevante para saber se a Fila está cheia.

Propriedade das Filas

Fila

Enfileira

X_{12}	X_{13}	X_{14}	X_{15}	X_{16}	X_{17}		X_8	X_9	X_{10}	X_{11}

tras frente

Posição “tras” fica sempre vazia para auxiliar a verificação de fila cheia.

A fila cheia é observada quando: *tras* + 1 é igual a *frente*. Esta verificação é obtida pela aritmética modular: *(tras + 1) % n == frente.*

Onde n é o tamanho do arranjo.

Propriedade das Filas

Fila

Enfileira

X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	

frente tras

Outra situação em que a fila é considerada cheia. Observe a situação onde a aritmética modula é necessária.

(tras + 1) % n == frente.

Implementação de Filas por meio de Arranjos

- O tamanho máximo do arranjo circular é definido por uma valor constante.
- Os outros campos da classe Fila contêm referências para a parte da frente e de trás da fila.
- Fila cheia e fila vazia:
  - Os apontadores ***frente*** e ***tras*** apontam para a mesma posição do círculo.

Implementação de Filas por meio de Arranjos

- Para distinguir as duas situações (fila cheia e fila vazia):
  - deixar uma posição vazia no arranjo.
  - Neste caso, a fila está cheia quando ***trás*+1 for igual a frente**.
- A implementação utiliza aritmética modular para enfileirar e desenfileirar (“**%**” em C e Java).

Implementação de Filas por meio de Arranjos

```c
#define MAX_TAM 1000  // Capacidade padrão

// Tipo do item armazenado na fila
typedef struct {
    int valor;
    // outros campos podem ser adicionados
} Item;

// Estrutura da fila
typedef struct {
    Item* itens;
    int frente;
    int tras;
    int capacidade;
} Fila;
```

Implementação de Filas por meio de Arranjos

```c
// Cria uma fila com capacidade padrão (1000)
Fila* criar_fila() {
    Fila* f = (Fila*) malloc(sizeof(Fila));
    if (!f) {
        fprintf(stderr, "Erro ao alocar memória para a fila\n");
        exit(1);
    }

    f->itens = (Item*) malloc(sizeof(Item) * MAX_TAM);
    if (!f->itens) {
        fprintf(stderr, "Erro ao alocar memória para os itens\n");
        exit(1);
    }

    f->capacidade = MAX_TAM;
    f->frente = 0;
    f->tras = 0;

    return f;
}

// Verifica se a fila está vazia
bool fila_vazia(Fila* f) {
    return f->frente == f->tras;
}

// Verifica se a fila está cheia
bool fila_cheia(Fila* f) {
    return f->tras == f->capacidade;
}
```

Implementação de Filas por meio de Arranjos

```c
// Enfileira um item
void enfileira(Fila* f, Item x) {
    if (fila_cheia(f)) {
        fprintf(stderr, "Erro: A fila está cheia\n");
        exit(1);
    }

    f->itens[f->tras++] = x;
}

// Desenfileira um item
Item desenfileira(Fila* f) {
    if (fila_vazia(f)) {
        fprintf(stderr, "Erro: A fila está vazia\n");
        exit(1);
    }

    return f->itens[f->frente++];
}

// Libera a memória da fila
void destruir_fila(Fila* f) {
    if (f) {
        free(f->itens);
        free(f);
    }
}
```

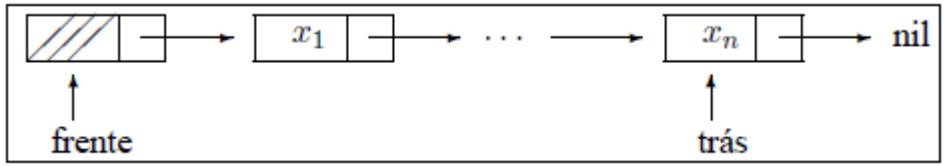
Implementação de Fila por meio de Estruturas Auto Referenciadas

Implementação de Filas por meio de Estruturas Auto Referenciadas

- Há uma célula *head* para facilitar a implementação das operações enfileira e desenfileira quando a fila está vazia.
- Fila vazia:
  - *frente* e *trás* referenciam para a mesma célula (*head*).
- Enfileirar um novo item:
  - Criar uma célula nova, ligá-la após a célula que contém x_n e colocar nela o novo item.

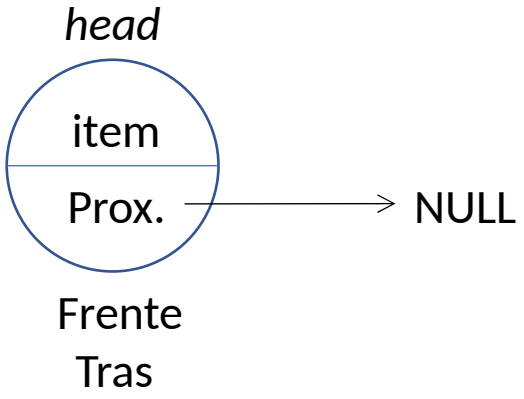
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Desenfileirar item x_1:
  - desligar a célula *head* da lista e a célula que contém x_1 passa a ser a célula *head*.



Implementação de Filas por meio de Estruturas Auto Referenciadas

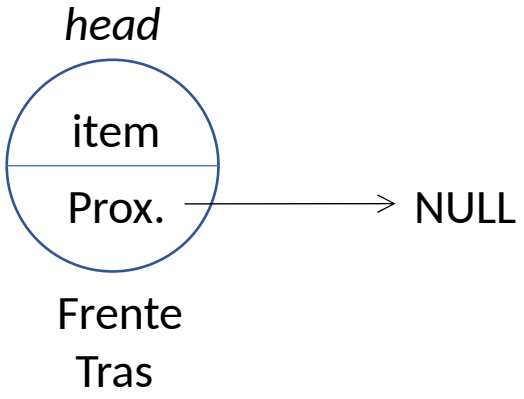
- Fila Vazia (frente e trás referenciam célula *head*)




```
    frente = (Celula*)
(malloc(sizeof(Celula*)));
    tras = frente;
    frente->prox = null;
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

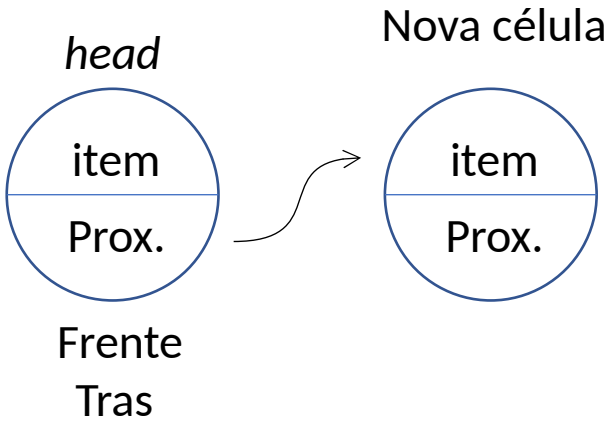
- Enfileira novo elemento na fila.




```
void enfileira (Item x) {
    tras->prox =
Celula*(malloc(sizeof(Celula*)));
    tras = tras->prox;
    tras->item = x;
    tras->prox = null;
  }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

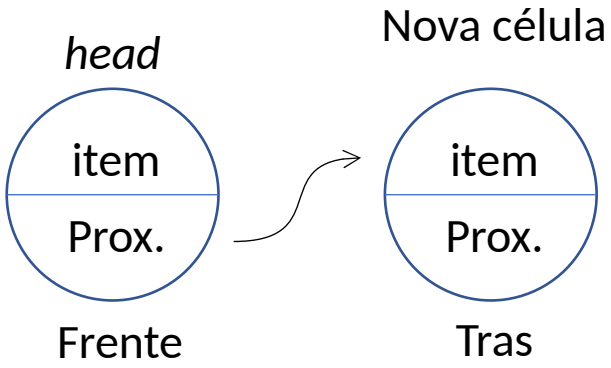
- Enfileira novo elemento na fila.




```
void enfileira (Item x) {
    tras->prox =
Celula*(malloc(sizeof(Celula*)));
    tras = tras->prox;
    tras->item = x;
    tras->prox = null;
  }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

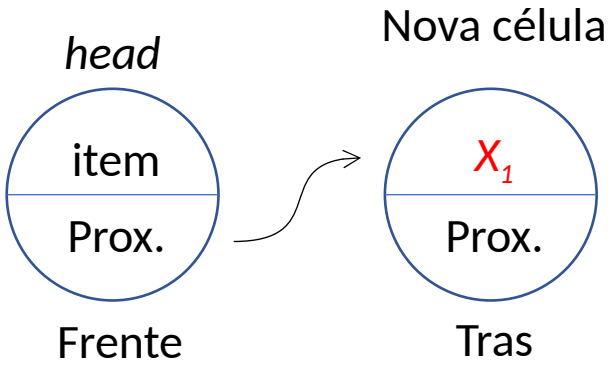
- Enfileira novo elemento na fila.




```
void enfileira (Item x) {
    tras->prox =
Celula*(malloc(sizeof(Celula*)));
    tras = tras->prox;
    tras->item = x;
    tras->prox = null;
  }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

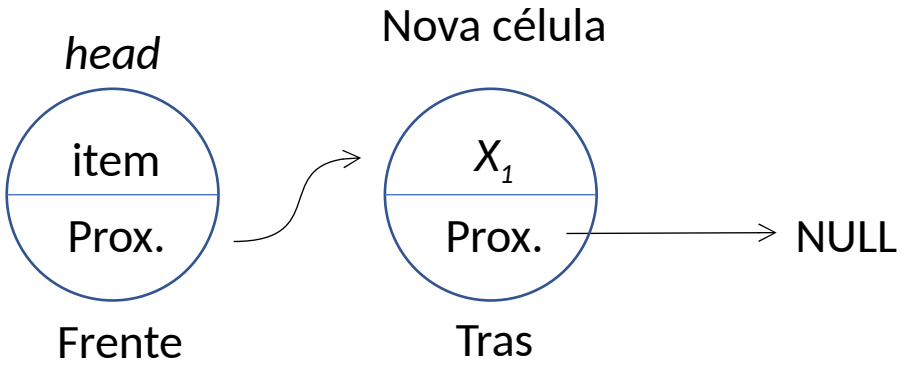
- Enfileira novo elemento na fila.




```
void enfileira (Item x) {
    tras->prox =
Celula*(malloc(sizeof(Celula*)));
    tras = tras->prox;
    tras->item = x;
    tras->prox = null;
  }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

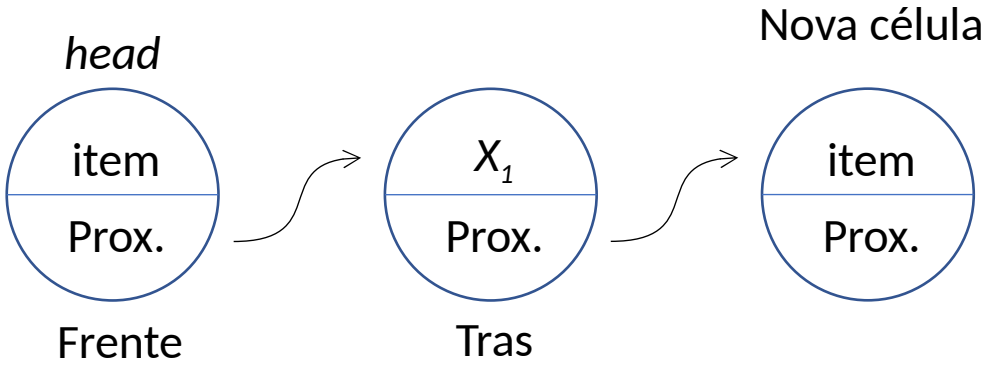
- Enfileira novo elemento na fila.




```
void enfileira (Item x) {
    tras->prox =
Celula*(malloc(sizeof(Celula*)));
    tras = tras->prox;
    tras->item = x;
    tras->prox = null;
  }
```

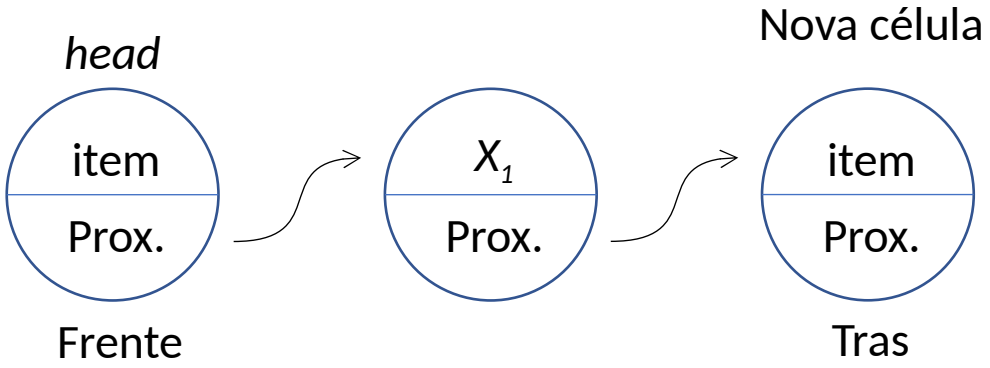
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Enfileira novo elemento na fila.



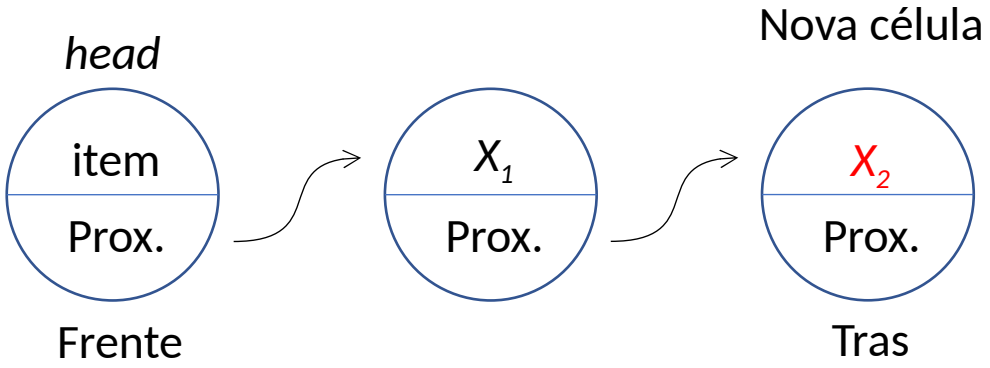
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Enfileira novo elemento na fila.



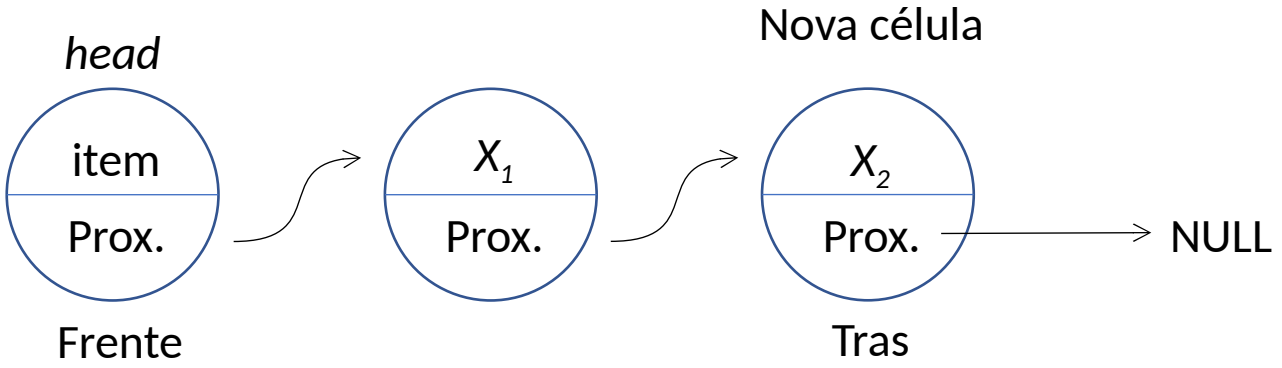
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Enfileira novo elemento na fila.



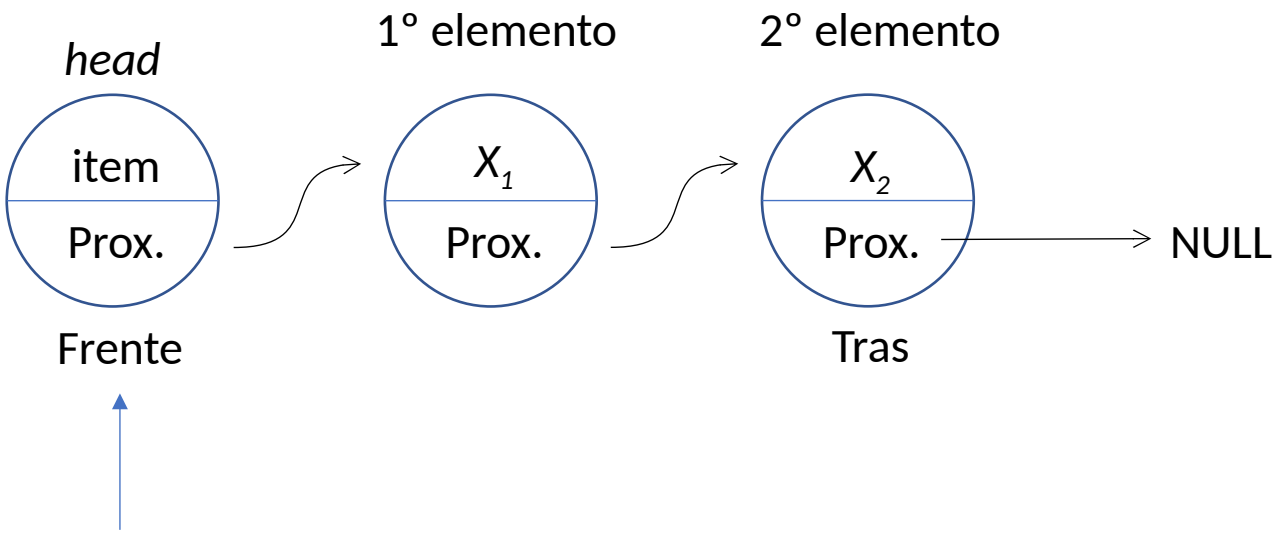
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Enfileira novo elemento na fila.



Implementação de Filas por meio de Estruturas Auto Referenciadas

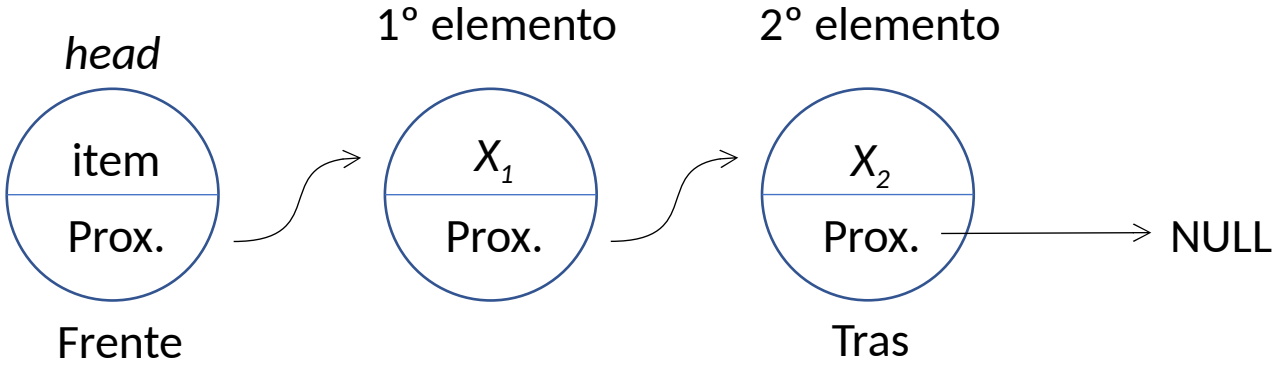
- Enfileira novo elemento na fila.



Observe que *head* é apenas uma célula para auxiliar as operações. O atributo "item" de *head* não armazena itens da fila.

Implementação de Filas por meio de Estruturas Auto Referenciadas

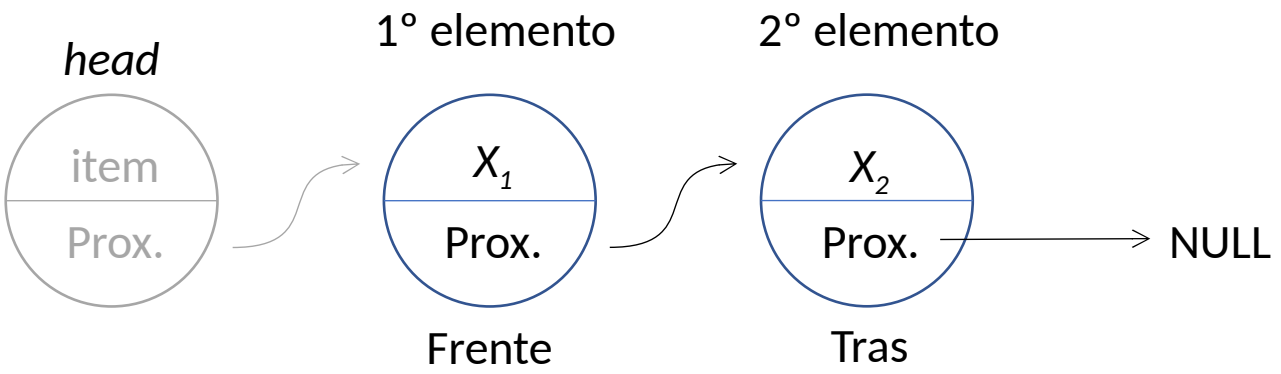
- Desenfileirar um item no inicio da fila.




```
Item desenfileira () {
   Celula *q = frente;
   if (vazia ())
     printf("Erro: A fila esta vazia");
   frente = frente->prox;
   free(q);
   Item item = frente->item; return item;
 }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

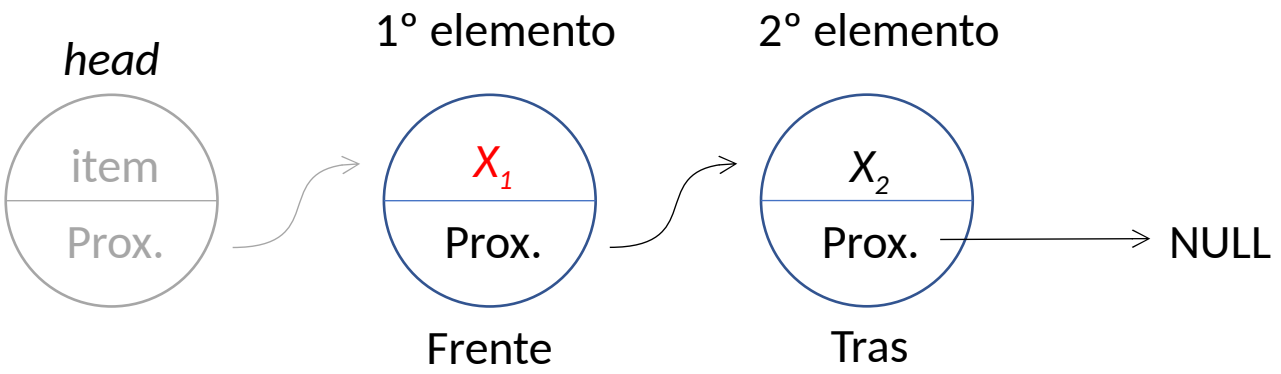
- Desenfileirar um item no inicio da fila.




```
Item desenfileira () {
   Celula *q = frente;
   if (vazia ())
     printf("Erro: A fila esta vazia");
   frente = frente->prox;
   free(q);
   Item item = frente->item; return item;
 }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

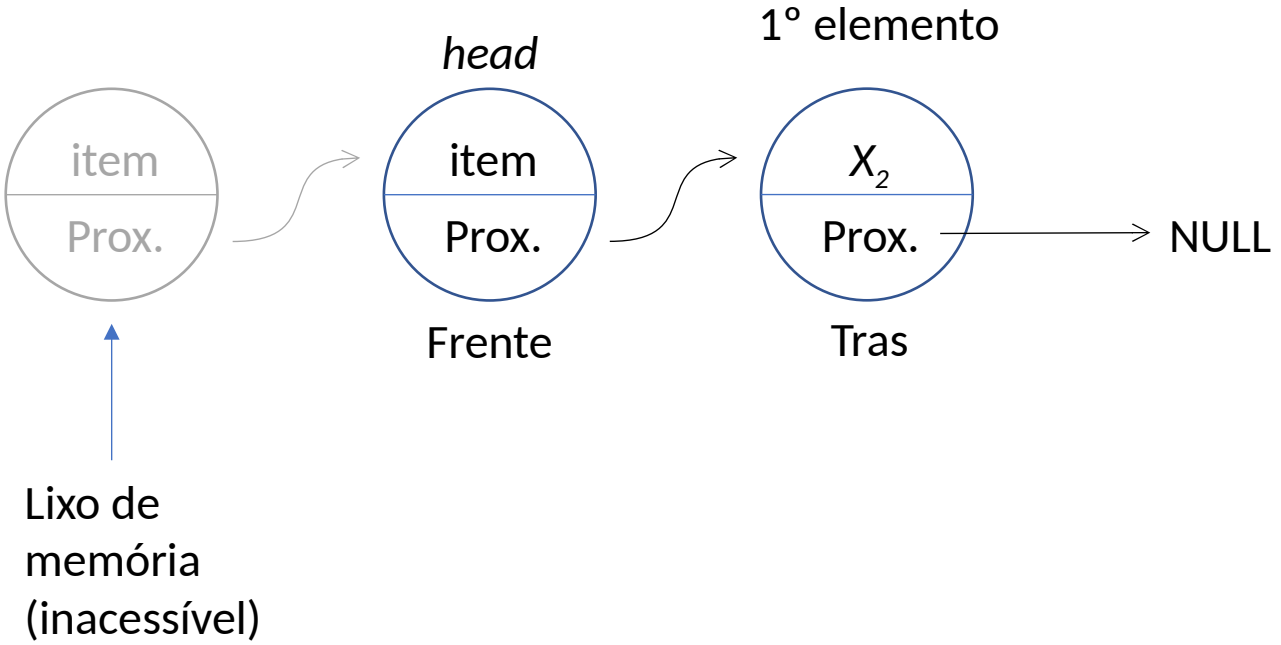
- Desenfileirar um item no inicio da fila.




```
Item desenfileira () {
   Celula *q = frente;
   if (vazia ())
     printf("Erro: A fila esta vazia");
   frente = frente->prox;
   free(q);
   Item item = frente->item; return item;
 }
```

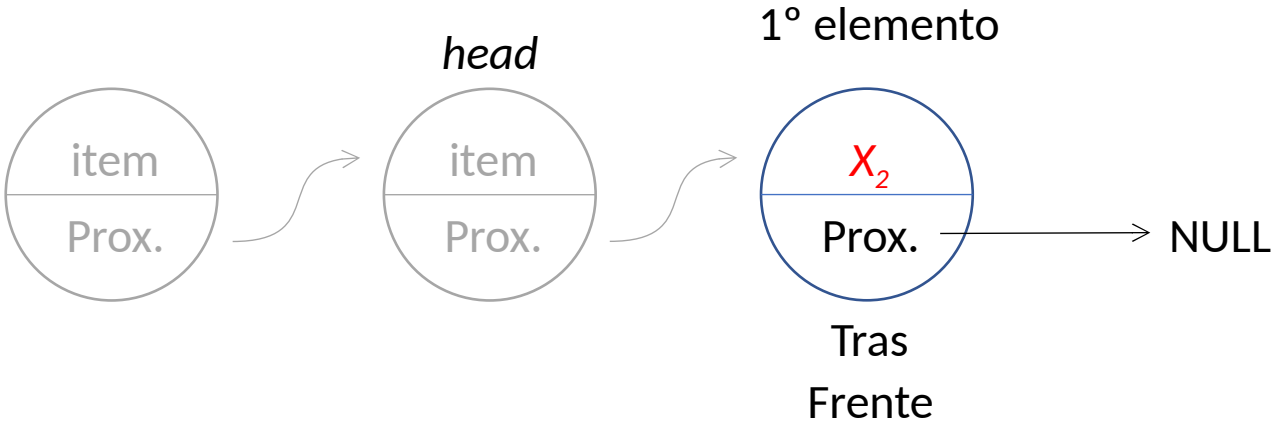
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Desenfileirar um item no inicio da fila.



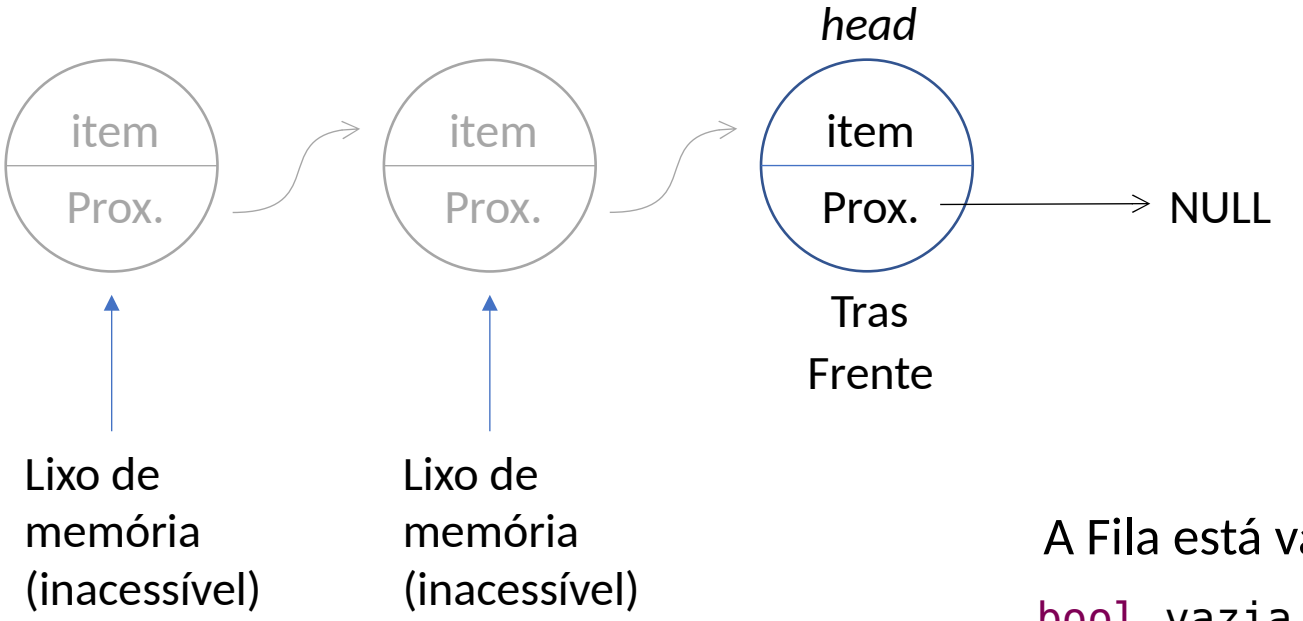
Implementação de Filas por meio de Estruturas Auto Referenciadas

- Desenfileirar um item no inicio da fila.



Implementação de Filas por meio de Estruturas Auto Referenciadas

- Desenfileirar um item no inicio da fila.



A Fila está vazia:

```
bool vazia () {
   return (frente == tras);
 }
```

Implementação de Filas por meio de Estruturas Auto Referenciadas

- A fila é implementada por meio de células.

- Cada célula contém um item da fila e uma referência para outra célula.

- A classe Fila contém uma referência para a frente da fila (célula *head*) e uma referência para a parte de trás da fila.

Deques

Deques

- Deque é uma generalização de uma Fila, em que as inserções e remoções de elementos podem ocorrer em qualquer uma das extremidades.

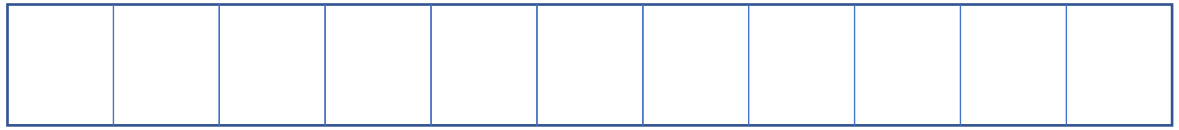
- Deque (**d**ouble **e**nded **que**ue)

- Para implementação por meio de vetores (arranjos) a estrutura circular, com na Fila, também é utilizada.

Deques (arranjos)

Fila

Deque Vazio



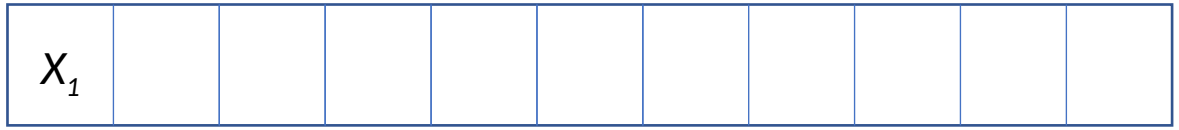
frente
tras

Conforme ocorre em uma Fila (regra de deque vazio: frente = tras)

Deques (arranjos)

Fila

Inserção no final (tras)



frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

X_1	X_2	X_3								

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

X_1	X_2	X_3	X_4							

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

X_1	X_2	X_3	X_4	X_5						

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

X_1	X_2	X_3	X_4	X_5	X_6					

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Inserção no final (tras)

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Remoção no início (frente)

	X_2	X_3	X_4	X_5	X_6	X_7				

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

Remoção no início (frente)

		X_3	X_4	X_5	X_6	X_7				

frente tras

Conforme ocorre em uma Fila

Deques (arranjos)

Fila

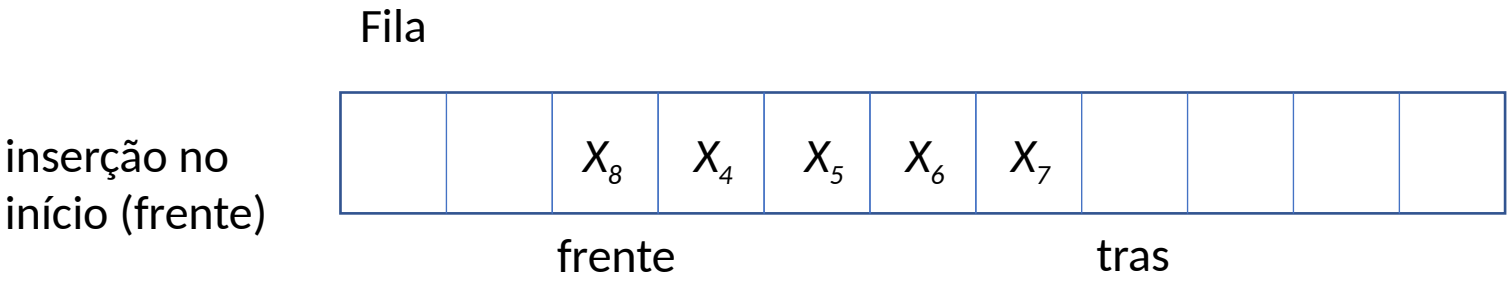
Remoção no início (frente)

			X_4	X_5	X_6	X_7				

frente tras

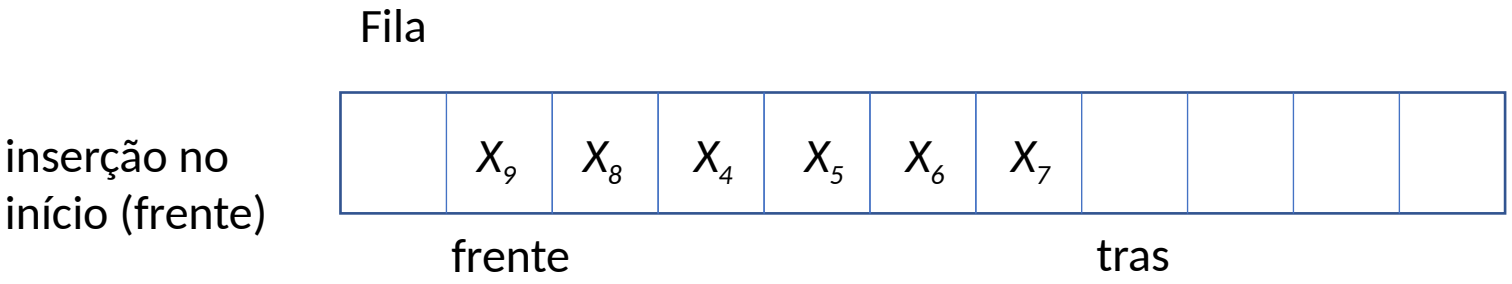
Conforme ocorre em uma Fila

Deques (arranjos)



Especialização de uma Fila (Deque). A inserção também pode ocorrer na extremidade “início”.

Deques (arranjos)



Especialização de uma Fila (Deque). A inserção também pode ocorrer na extremidade “início”.

Deques (arranjos)

Fila

inserção no
início (frente)

X_{10}	X_9	X_8	X_4	X_5	X_6	X_7				

frente tras

Especialização de uma Fila (Deque). A inserção também pode ocorrer na extremidade “início”.

Deques (arranjos)

Fila

inserção no
início (frente)

X_{10}	X_9	X_8	X_4	X_5	X_6	X_7				X_{11}

tras frente

Especialização de uma Fila (Deque). A inserção também pode ocorrer na extremidade “início”.

Observe que a estrutura também é circular (como a Fila)

Deques (arranjos)

Fila

inserção no
início (frente)

X_{10}	X_9	X_8	X_4	X_5	X_6	X_7			X_{12}	X_{11}

tras frente

Especialização de uma Fila (Deque). A inserção também pode ocorrer na extremidade “início”.

Deques (arranjos)

Fila

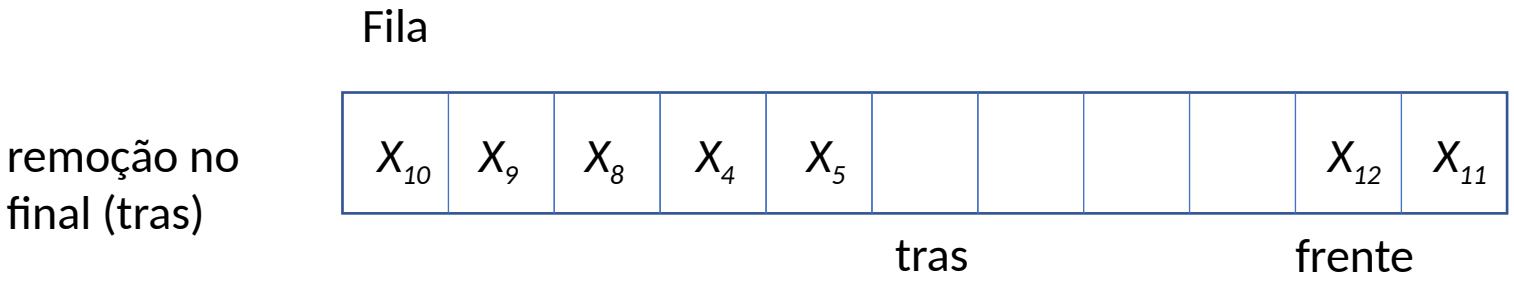
remoção no final (tras)

X_{10}	X_9	X_8	X_4	X_5	X_6				X_{12}	X_{11}

tras frente

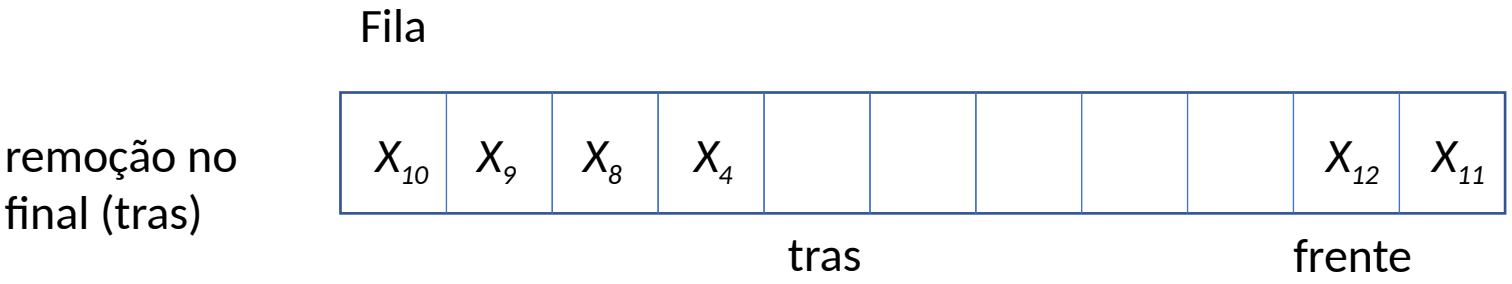
Especialização de uma Fila (Deque). A remoção também pode ocorrer na extremidade “final”.

Deques (arranjos)



Especialização de uma Fila (Deque). A remoção também pode ocorrer na extremidade “final”.

Deques (arranjos)



Especialização de uma Fila (Deque). A remoção também pode ocorrer na extremidade “final”.

Deques (arranjos)

Fila

Deque cheio

X_{10}	X_9	X_8	X_4	X_{13}	X_{14}		X_{16}	X_{15}	X_{12}	X_{11}

tras frente

Posição “tras” fica sempre vazia para auxiliar a verificação de Deque cheio.

Como na Fila, o Deque cheio é observada quando: *tras* + 1 é igual a *frente*. Esta verificação é obtida pela aritmética modular: *(tras + 1) % n == frente.*

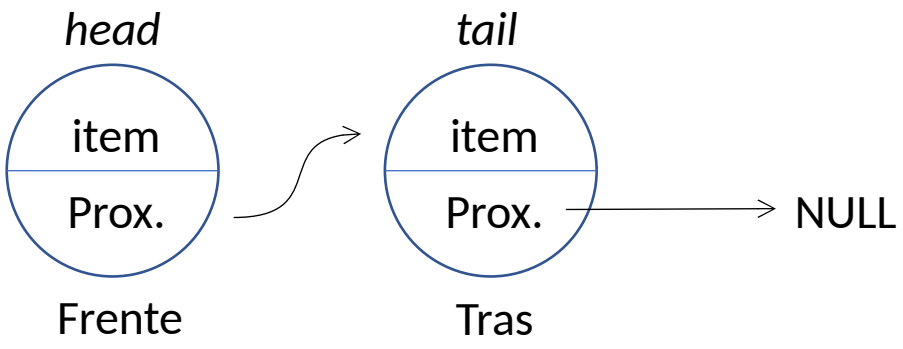
Onde n é o tamanho do arranjo.

Deques (lista auto referenciadas)

- Em um Fila com implementação por meio de estruturas auto referenciadas, há uma célula *head* (cabeça) para facilitar a implementação das operações enfileira e desenfileira quando a fila está vazia.
- Em um Deque, além da célula *head*, há a célula *tail* (calda) para auxiliar as operações de remoção e inserção no final do Deque (também conhecido como *head-tail linked list*).
- Deque vazio:
  - *Frente (head)* referencia a célula *trás (tail)*.

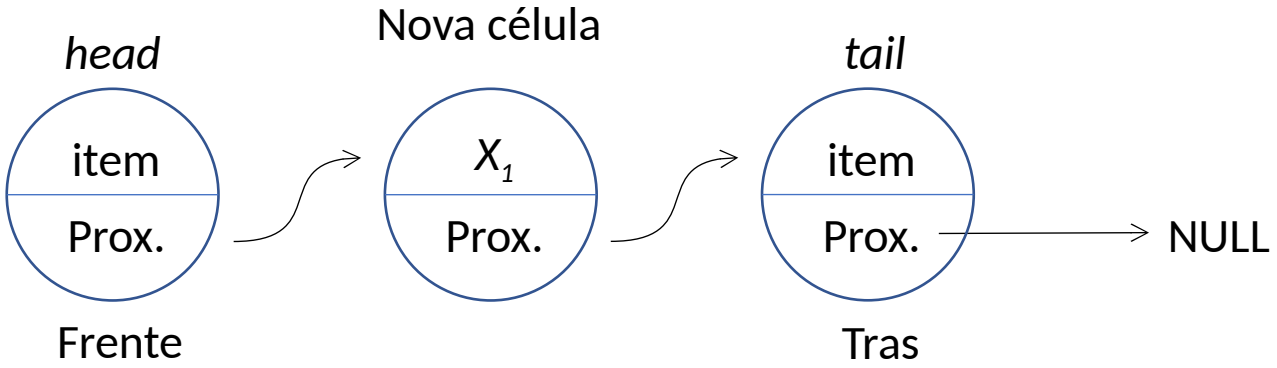
Deques (lista auto referenciadas)

- Deque inicialmente vazio.



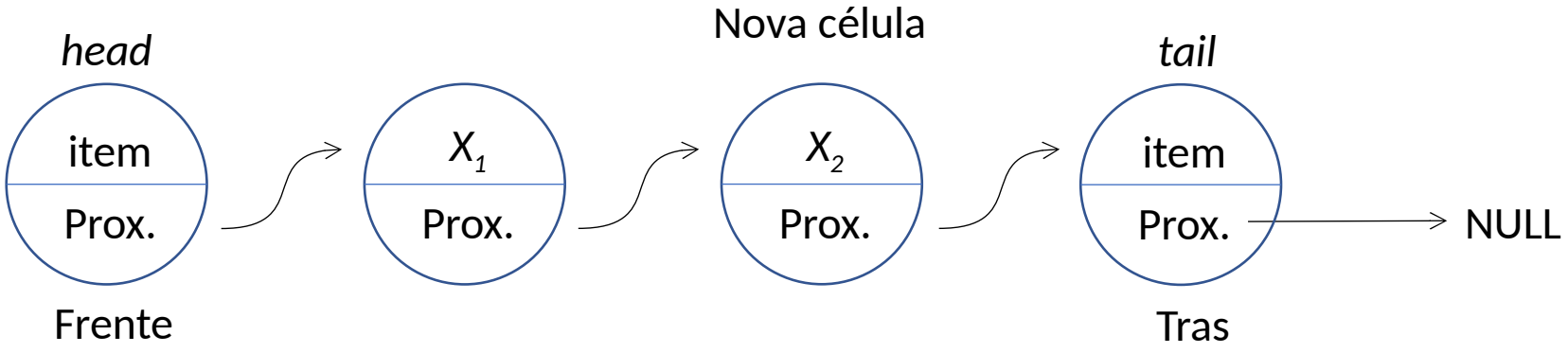
Deques (lista auto referenciadas)

- Inserção de um elemento no final (tras)



Deques (lista auto referenciadas)

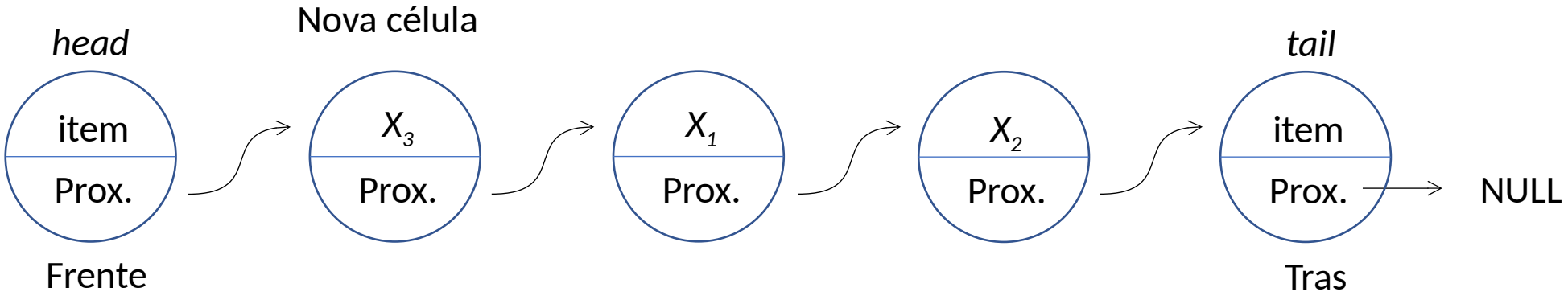
- Inserção de um elemento no final (tras)



A operação que insere no final de um Deque deve percorrer toda a lista a partir do elemento "*frente*" até encontrar o elemento que referencia "*tras*" (em um Deque vazio, este elemento é o próprio "*frente*"). Então, esta célula que referencia "*tras*" deve referenciar um nova célula que, por sua vez, passa a referenciar "*tras*".

Deques (lista auto referenciadas)

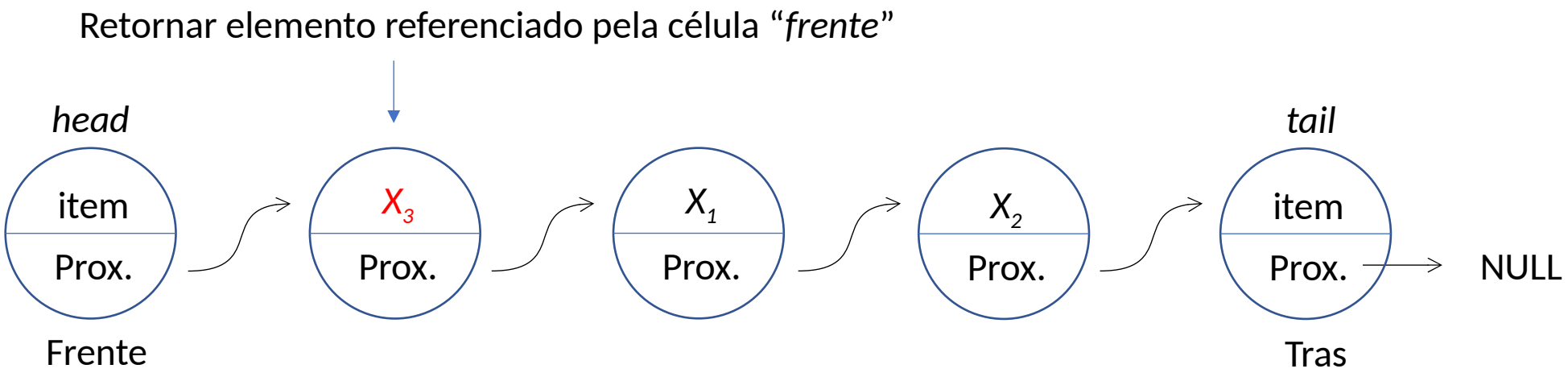
- Inserção de um elemento no início (frente)



A operação que insere no início de um Deque deve inserir uma nova célula a partir do elemento "*frente*". Então, a célula "*frente*" deve referenciar um nova célula que, por sua vez, passa a referenciar a próxima célula (em um Deque vazio será a célula "*tras*").

Deques (lista auto referenciadas)

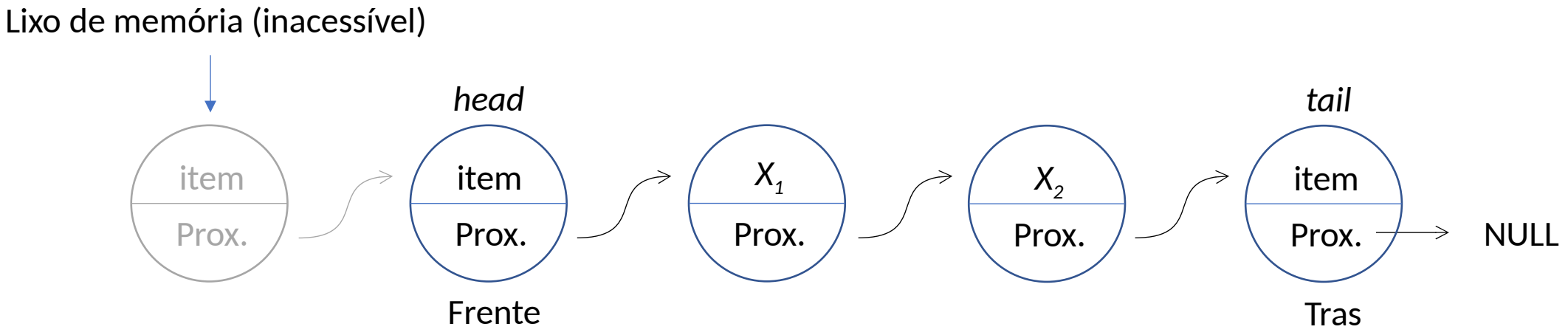
- Remoção de um elemento no início (frente)



A operação que remove elemento de um Deque no início é semelhante a operação de *desenfileira* de uma Fila.
Atenção apenas se para indicação de Deque vazio, onde "*frente*" passa a referenciar a célula "*tras*".

Deques (lista auto referenciadas)

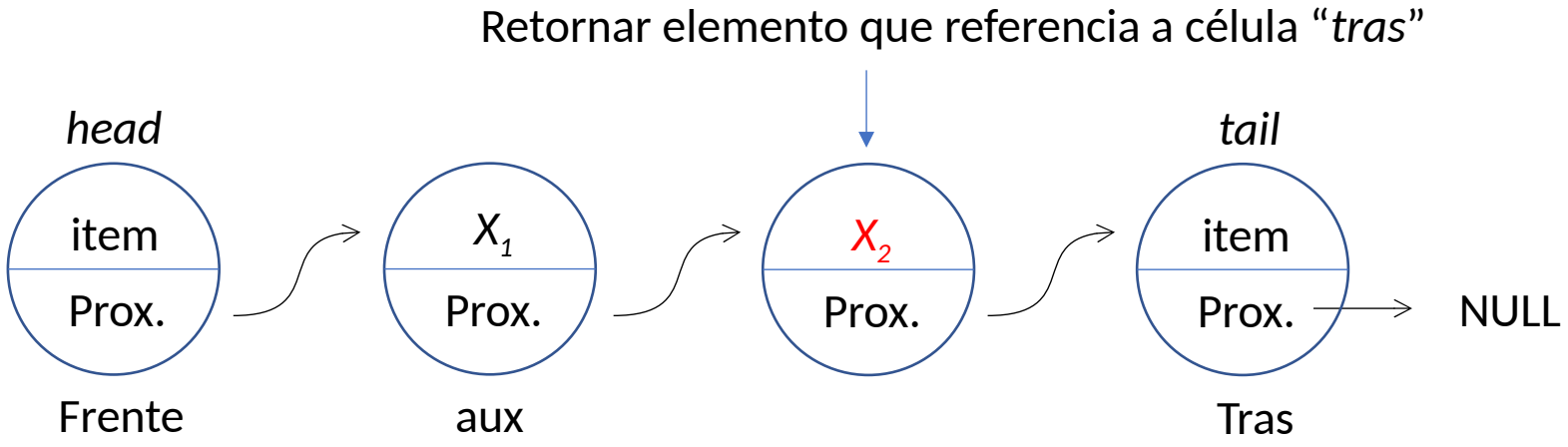
- Remoção de um elemento no início (frente)



A operação que remove elemento de um Deque no início é semelhante a operação de *desenfileira* de uma Fila.
Atenção apenas se para indicação de Deque vazio, onde "*frente*" passa a referenciar a célula "*tras*".

Deques (lista auto referenciadas)

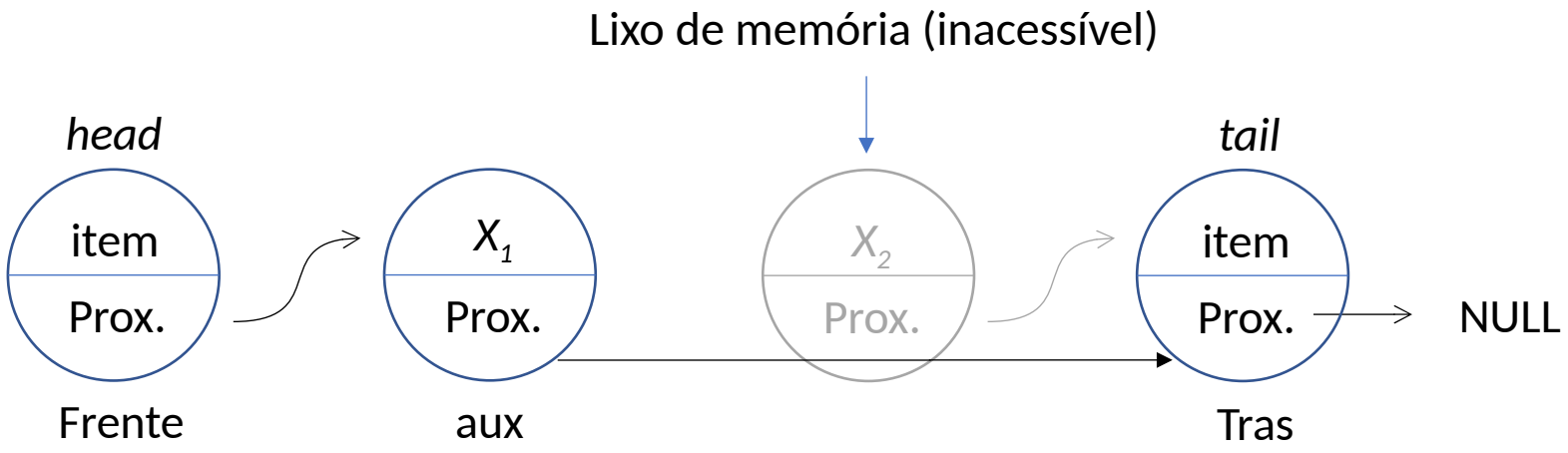
- Remoção de um elemento no final (tras)



A operação que remove elemento de um Deque no final deve percorrer toda a lista a partir do elemento "*frente*" até encontrar o elemento que referencia "*tras*" e retornar o item deste elemento. Então, a célula "aux" passa a referenciar a célula "*tras*"

Deques (lista auto referenciadas)

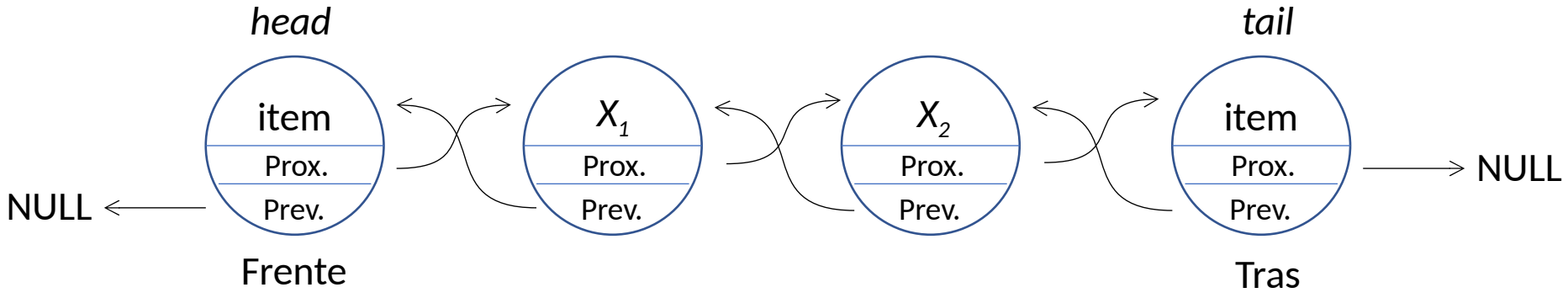
- Remoção de um elemento no final (tras)



A operação que remove elemento de um Deque no final deve percorrer toda a lista a partir do elemento "*frente*" até encontrar o elemento que referencia "*tras*" e retornar o item deste elemento. Então, a célula "aux" passa a referenciar a célula "*tras*"

Deques (lista duplamente ligadas)

- Remoção e inserção na extremidade "*tras*" são mais eficientes pois não precisa percorrer toda a lista até encontrar a referência para a célula "*tras*". Torna-se uma solução melhor para Deques.



Exercícios

- Implemente e teste a classe Fila (com vetor e com listas ligadas) estudada nesta aula.

- A partir da classe Fila, Implemente a classe Deque (com vetor e com listas ligadas). Devem ser feitas algumas modificações nos métodos existentes da Fila e a implementação de dois novos métodos para inserir na extremidade "*frente*" e remover na extremidade "*tras*".

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).